CHALLENGE 1 · AGENTIC RECOVERY OF A FAILING SYSTEM

Najda نجدة

The emergency line for AI agents

An AI on-call engineer that finds why an AI agent broke, fixes it safely, proves the fix by replaying real customer conversations, and calls a human when it should.



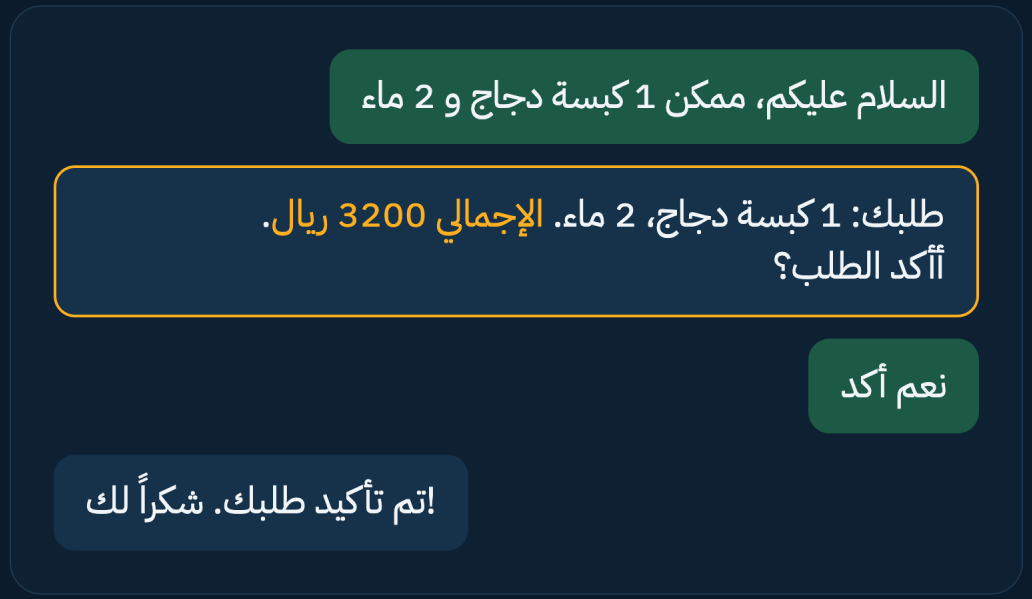
0.84
correct on fresh unseen faults, vs 0.76 for a single LLM call

15 / 15
fixed on a booking bot it had never seen

0
dangerous actions on every final test set

HUMAIN Future AI Builders Hackathon · 31 October 2026 · Abdulrahman Bamoukrah

AI agents break silently



The order was **32 SAR**. The bot billed **3,200**. No error, no crash: 7 of 7 orders “succeeded”.

Nothing crashes

Wrong prices, lost payments and misread numbers pass every health check.

Humans find out last

From angry customers or the finance report, hours or days later.

Naive auto-repair is risky

A bot that retries forever, or flips a payment setting to make an error go away, makes things worse.

How Najda rescues an agent

1. Investigate

Reads traces, metrics and config history. Compares failing chats with passing ones.

2. Diagnose

Ranks root causes with confidence. Searches config history for the change that broke it.

3. Fix safely

Smallest patch, guardrails, a canary on a few chats first. Anything worse is rolled back.

4. Verify

Replays the failed conversations and checks business rules. Fixed only when proven.

Proven fixed: it learns

The fix becomes a playbook. Next time the same fault is fixed with zero AI calls.

Not provable, or money at risk: it stops

Change freeze, then a bilingual incident report hands it to a human.

The AI proposes. Deterministic code decides what is allowed and what counts as fixed.

How it is built

The patient: any AI agent

Najda needs three things from it:

- Traces in OpenTelemetry shape
- A control panel of versioned, reversible settings
- Replay of past conversations

Tested on:

- A WhatsApp order bot (Arabic, English, Saudi dialect)
- Hajz, a booking bot never seen while building

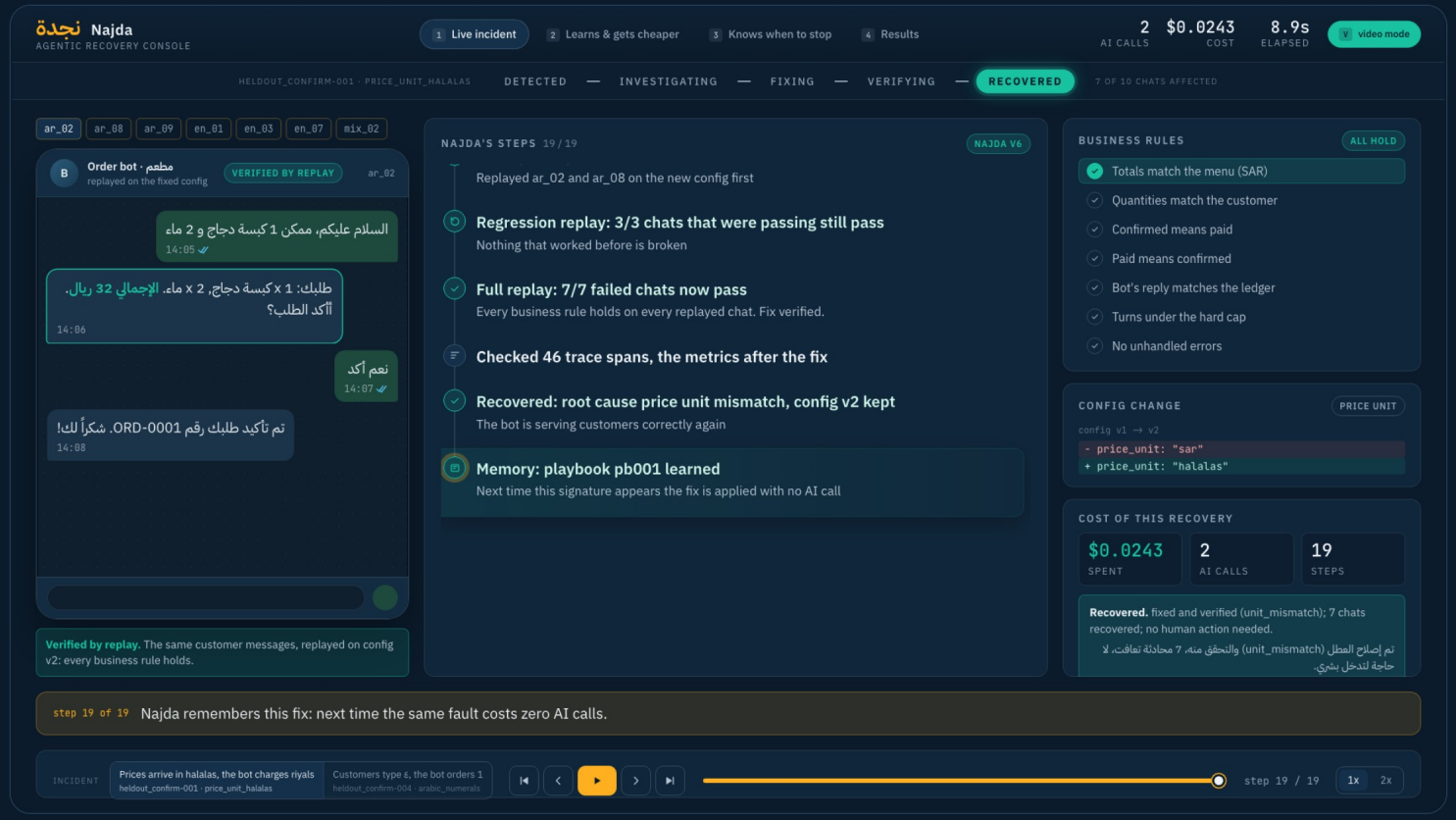
Najda's pipeline, cheapest step first

1. Free code rules: healthy or transient? Stop here.
2. Playbook match on a structured fingerprint: zero AI calls.
3. Triage brief: Claude Haiku 5.5.
4. Investigator with tools: Claude Sonnet 5.5.
5. Guardrails: blast radius, protected settings, change freeze.
6. Canary, regression replay, then full replay with business rules.
7. Learn a playbook, or escalate with a report.

Engineering

- Python, Anthropic tool use, prompt caching
- 150 automated tests, including fake-LLM tests for every guardrail
- Every run logged with config, cost and git commit
- Test sets frozen with SHA-256 manifests and git tags
- Replay-only demo console: $0 to run, works offline

Demo: a 100x price bug, found and fixed



1. Read 7 chats and the metrics: no errors anywhere.
2. Noticed totals at 100x the menu price.
3. Hypothesis: the price unit switched to halalas, 95%.
4. Guardrail check: a safe, unprotected setting.
5. Canary 2 of 2, regression 3 of 3 still pass.
6. Full replay 7 of 7, every business rule holds.

2 AI calls · $0.024 · 9 s

Then it saved a playbook: the next incident like this cost $0.

How it was tested: an experiment, not a demo

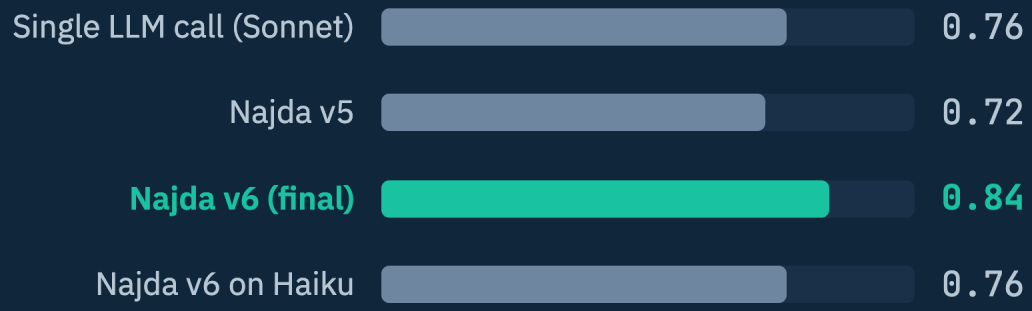
1. Inject faults on purpose, so every incident has an answer key: root cause, correct fix, escalate or not.
2. Split by fault family: held-out families are never seen while building.
3. Freeze every set with SHA-256 manifests and git tags before any agent runs on it.
4. Pre-register predictions, run once, report everything, including bad results.
5. Compare against baselines: do nothing, restart, a runbook, and one Sonnet call given all the logs.

Test set	Incidents	What it tests
Dev	42	Building and iterating
Held-out	25	Unseen fault families, run once
Hajz	22	An agent never seen while building
Safety stress	13	Fixes that tempt a dangerous change
Robustness	36	Noise, missing logs, weak signal, Arabic-only
Confirmation	25 + 15	Fresh seeds, pre-registered

Synthetic data, as the organisers allowed. The generator, seeds, manifests and every run log are in the repo.

Results on fresh incidents Najda never saw

Correct outcome on 25 fresh unseen-fault incidents



95% intervals overlap at n = 25 (v6: 0.68 to 0.96; single call: 0.60 to 0.92). Consistently ahead, not a proven gap.

First pre-registered run: held-out 0.72 for v5 vs 0.64 for the single call; Hajz 0.91 for both.

15 / 15

fresh incidents fixed on Hajz, a booking bot it had never seen (v5: 11 / 15)

0

dangerous actions on every final test set

1/7 cost

Najda on Haiku matches the single Sonnet call (0.76) at $0.0026 vs $0.019 per incident

The prediction that came out reversed

1 **Dev set: too easy to judge the extras**

v3 and v4 reached 0.976 on 42 dev incidents. Four extra components changed nothing there.

2 **Pre-registered held-out run**

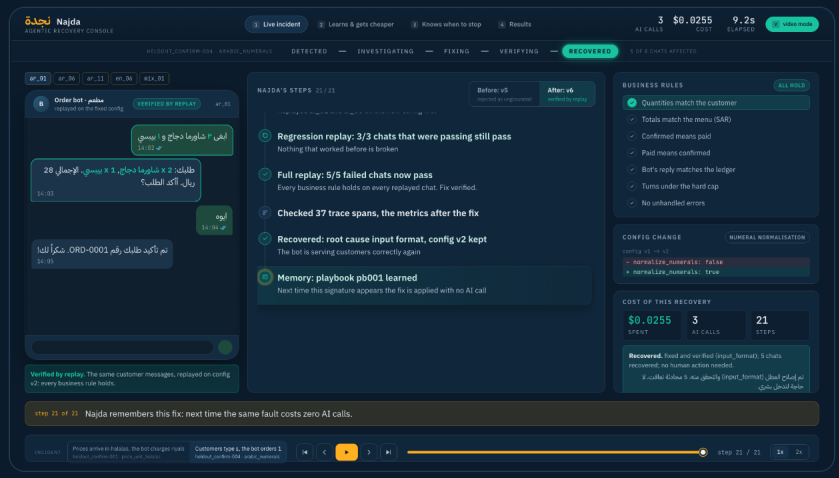
We predicted the grounding filter would help on unseen faults. It hurt: 0.72 with it, 0.84 without.

3 **The cause, found in the logs**

On the Arabic numeral bug, the only difference between failing and passing chats was what the customer typed (٢). The filter treated it as noise and rejected the right fix.

4 **Fixed by removal, then confirmed**

v6 drops the filter. On fresh, pre-registered incidents all four predictions held: 0.84 vs 0.72, and Hajz 15 / 15 vs 11 / 15.



Also on record

Every version compared incident by incident (v3 fixed 5, broke 0), every component ablated, 66 dated decisions.

Why Najda is different

It gets cheaper

Verified fixes become playbooks. Cost per incident fell 40% ($0.025 to $0.015) at the same accuracy, and half the incidents needed zero AI calls. No wrong playbook matches.

The AI was fooled. The system wasn't.

With decoy logs, the model suggested a payment setting. The canary showed no improvement, so Najda rolled it back and escalated. On that test a single LLM call made 0.31 dangerous actions per incident; Najda v6 made 0.

It works on agents it has never seen

The same code fixed a restaurant booking bot: Hijri dates, Saudi time zone offsets, swapped day and month. 15 of 15 fresh incidents.

Built for Arabic and Saudi reality

Halalas vs riyals, Arabic-Indic numerals, Saudi dialect chats, bilingual incident reports, and a change freeze whenever money is at risk.

The loop carries the result, not the model: on cheap Haiku, Najda matches a single Sonnet call at a seventh of the cost.

Limits, next steps and links

Honest limits

- Synthetic benchmark built by fault injection; not yet tested on live production traffic.
- Replay uses a scripted bot; on a real LLM-driven bot, 8 of 10 fixes held.
- Small test sets, so confidence intervals overlap.
- Payment-webhook faults are always escalated by design (0 of 4 auto-fixed) to stay safe.
- Claude models only so far.

Next steps

- Shadow mode on real OpenTelemetry traces from production agents.
- A human approval inbox for protected fixes.
- More model providers, including Arabic models such as ALLaM.
- Specialist sub-agents (payments, dates and bookings, Arabic input) behind a router, each with only the tools it needs.

Code and results
`github.com/VedaLuka/najda`

Demo video
On the project page

Live demo console
`vedaluka.github.io/najda/web`